

FinResearch AI — Multi-Agent Financial Analysis System

Complete Project Documentation (Beginner-Friendly)

TABLE OF CONTENTS

- 1. Project Overview
 - 2. What is This Project?
 - 3. System Architecture
 - 4. Multi-Agent Pipeline Explained
 - 5. Technology Stack
 - 6. Database Schema
 - 7. API Reference
 - 8. Setup & Installation Guide (Local)
 - 9. Deployment Guide
 - 10. How to Use the Platform
 - 11. Troubleshooting
-

1. PROJECT OVERVIEW

FinResearch AI is a full-stack web application that uses multiple AI agents working together to analyze financial documents like annual reports (10-K filings), earnings transcripts, and investor presentations.

Target Users: Finance students, MBA candidates, early-career analysts

Key Problem Solved: Manually extracting key metrics, identifying risks, and comparing companies across complex financial documents requires significant time, effort, and domain expertise. This system automates that workflow using specialized AI agents.

2. WHAT IS THIS PROJECT? (For Beginners)

What is a 10-K Filing?

A 10-K is a comprehensive annual report that publicly traded companies must file with the SEC (Securities and Exchange Commission). It contains:

- Company overview and business description
- Financial statements (income statement, balance sheet, cash flows)
- Risk factors
- Management's discussion and analysis (MD&A)
- Notes to financial statements

What is a Multi-Agent AI System?

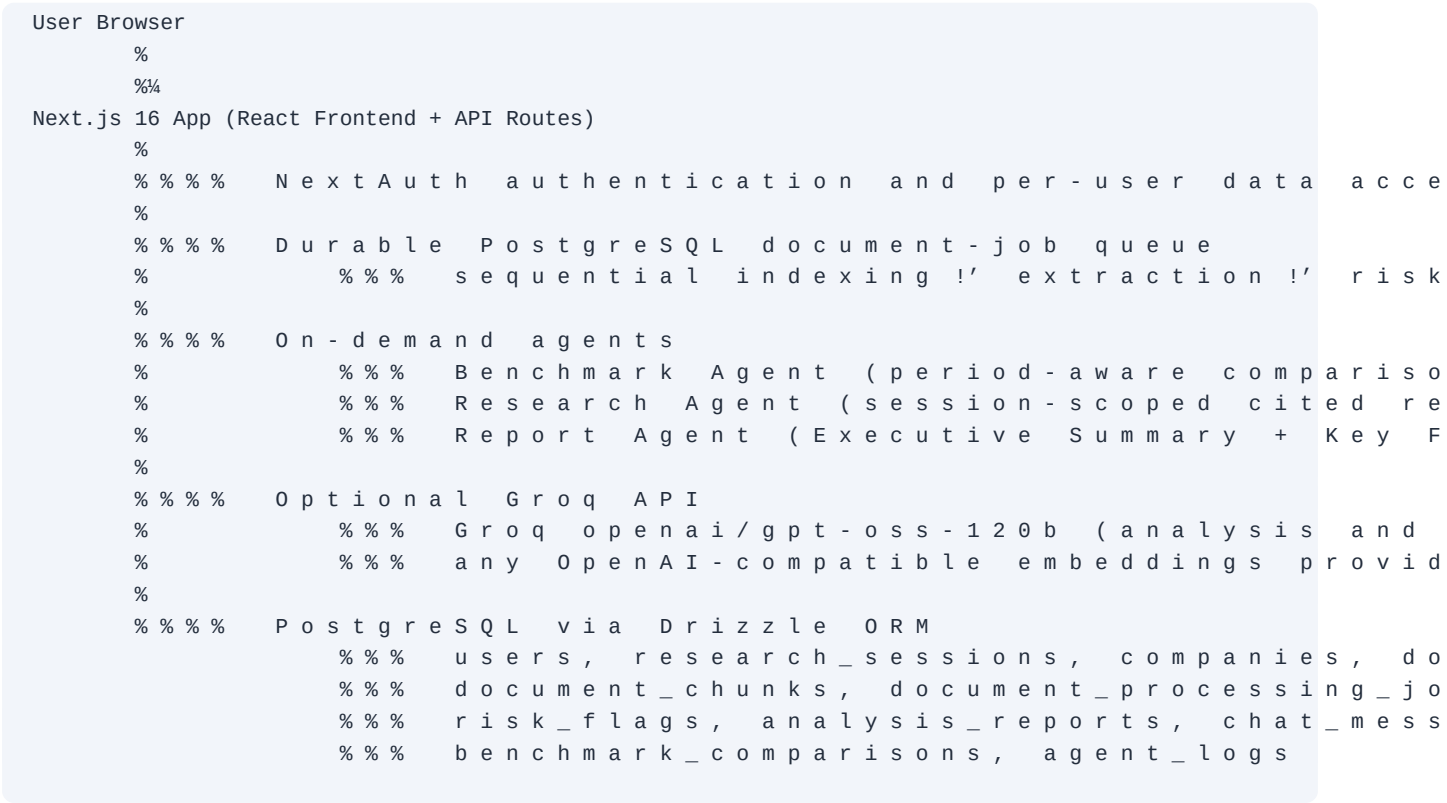
Instead of having one AI do everything, we have specialized AI agents, each expert in one task:

- Like a real investment bank where different analysts handle research, risk, and writing

What Does This System Do?

1. You upload a financial document (or use pre-loaded ones)
2. A durable job runs document indexing, metric extraction, risk checks, and optional embeddings in order
3. You get source-checked metrics, evidence-backed risk signals, and period-aware comparisons
4. You can ask questions in plain English and get cited answers

3. SYSTEM ARCHITECTURE



4. MULTI-AGENT PIPELINE EXPLAINED

Agent 1: Document Agent

What it does: Takes uploaded financial documents and processes them for AI analysis

Steps:

1. Receives file (PDF, DOCX, or TXT)
2. Extracts text content
3. Cleans and normalizes text
4. Splits into ~1500-character chunks with 200-char overlap
5. Identifies document sections (Business Overview, Risk Factors, etc.)
6. Stores source chunks with section labels; page numbers remain blank when the parser cannot verify them
7. Starts the sequential, recoverable processing stages

Why chunking? AI models have token limits. Breaking documents into smaller chunks lets us search and retrieve only relevant sections.

Agent 2: Extraction Agent

What it does: Reads financial documents and extracts structured metrics

Extracts:

- Revenue, net income, gross profit
- Margins: gross, operating, net, EBITDA
- Balance sheet: assets, liabilities, equity, cash, debt
- Ratios: current ratio, D/E ratio, ROE, ROA
- Cash flows: operating, capital expenditures, free cash flow
- Per share: EPS

How: Uses Groq JSON extraction when configured, validates each metric against an exact quote and matching numeric evidence, and fills supported values with a deterministic local parser when needed.

Agent 3: Risk Agent

What it does: Combines deterministic disclosure checks, source-matched metric anomalies, and optional model findings.

Checks include audit/going-concern language, accounting disclosures, liquidity warnings, balance-sheet anomalies, and cross-period debt, revenue, or margin movements. Trend checks require comparable periods and source evidence whose values match the stored metrics. Model-added findings are retained only when source quotes and numeric claims validate. These are screening signals, not audit conclusions.

Agent 4: Embedding Agent

What it does: Optionally creates semantic vectors for document chunks using Groq embeddings. Vectors are stored in JSONB with a model identifier; keyword search remains available if

embeddings are not configured or compatible.

Agent 5: Benchmark Agent

What it does: Compares accessible companies using their latest stored financial metrics and risk flags. It displays source documents and fiscal periods and warns when periods are missing or differ. The system does not generate buy/sell recommendations.

Agent 6: Research Agent (Conversational)

What it does: Retrieves evidence from the signed-in user's selected research session. Session ownership is checked before reading history, writing messages, or searching documents.

Each generated factual claim must use a retrieved citation and exact quote. Numeric values, currencies, scales, and percentage units must match the quote. When Groq is unavailable, the agent returns relevant retrieved excerpts rather than canned statistics. Quote checks improve traceability but do not prove semantic entailment; verify important conclusions against the original filing.

Agent 7: Report Agent

What it does: Creates an Executive Summary and a deterministic Key Financials table with periods and source filenames, then adds detailed analysis when Groq is available. The table remains available in the local report fallback.

5. TECHNOLOGY STACK

Frontend

- Next.js 16 with App Router — React framework for server-side rendering
- React 19 — UI component library
- Tailwind CSS 4 — Utility-first CSS framework
- Recharts — Charts and data visualization
- Lucide React — Icon library
- react-hot-toast — Notification toasts
- react-dropzone — File drag-and-drop upload

Backend

-
- Next.js API Routes — Serverless API endpoints
 - NextAuth.js — Authentication (JWT sessions)
 - Drizzle ORM — Type-safe database ORM
 - bcryptjs — Password hashing
 - pdf-parse — PDF text extraction
 - mammoth — DOCX text extraction

AI

-
- Groq API (@google/generative-ai), optional for model analysis
 - openai/gpt-oss-120b: configurable Groq default for extraction, risk review, research, benchmarking, and reports
 - Optional embeddings provider (EMBEDDING_*): semantic retrieval; Groq itself has no embeddings API

Database

-
- PostgreSQL — Relational database
 - 12 schema tables, including durable document jobs and optional embeddings

Infrastructure

-
- Node.js 20.9+ — Server runtime
 - npm — Package manager
-

6. DATABASE SCHEMA

users

Stores user accounts with hashed passwords.

research_sessions

Named workspaces where users organize their analysis work. Each session can contain multiple documents and chat conversations.

companies

Company profiles (name, ticker, sector, industry). Pre-seeded with Apple, Microsoft, Tesla, Amazon.

documents

Uploaded financial documents with metadata, ownership, fiscal period, processing status, and embedding status.

document_chunks

Split chunks from each document for searchable retrieval. Each chunk is approximately 1500 characters with section labeling; optional vectors and model names are stored in JSONB.

document_processing_jobs

Durable, retryable processing jobs with attempts, due times, stale locks, and last-error details.

financial_metrics

Extracted financial numbers linked to both document and company. Includes 25+ metric fields.

risk_flags

Risk items identified by the Risk Agent. Includes severity, source text, and recommendations.

analysis_reports

Generated research reports with executive summary and full analyst write-up.

chat_messages

Research Agent conversation history with citations, scoped to a user-owned session.

benchmark_comparisons

Comparison metadata schema for research sessions.

agent_logs

Audit trail of all agent activities (for debugging and transparency).

7. API REFERENCE

Authentication

-
- POST /api/auth/register — Create new user account
 - POST /api/auth/signin — Sign in (NextAuth)
 - POST /api/auth/signout — Sign out

Research Sessions

-
- GET /api/sessions — List all user sessions
 - POST /api/sessions — Create session
 - GET /api/sessions/[id] — Get session details
 - PATCH /api/sessions/[id] — Update session
 - DELETE /api/sessions/[id] — Delete session

Documents

-
- GET /api/documents — List the signed-in user's documents
 - POST /api/documents — Upload a document and atomically enqueue a durable job
 - GET /api/documents/[id] — Get an accessible document with metrics, risks, chunks, and activity
 - POST /api/documents/[id]/embeddings — Start optional semantic indexing
 - DELETE /api/documents/[id] — Delete an accessible document
 - GET /api/worker/documents — Recover one due job; requires Authorization: Bearer \$CRON_SECRET

Companies

-
- GET /api/companies — List companies and latest metrics visible to the signed-in user (including seeded demo data)

Financial Metrics

-
- GET /api/metrics — Get visible metrics (optional companyIds=id1,id2 or documentId)

Risk Analysis

-
- GET /api/risks — Get visible risk flags (optional companyIds=id1,id2, documentId, or severity)

Benchmarking

-
- POST /api/benchmark — Compare accessible companies, latest periods, source documents, and risks (body: { companyIds: string[], sessionId? })

Reports

-
- GET /api/reports — List reports
 - POST /api/reports — Generate new report
 - GET /api/reports/[id] — Get full report
 - DELETE /api/reports/[id] — Delete report

Chat/Research Agent

-
- GET /api/chat?sessionId=xxx — Read history only if the signed-in user owns the session
 - POST /api/chat — Ask a session-scoped research question (body: { sessionId, content }); failures return an error, not a canned answer

Dashboard

-
- GET /api/dashboard — Get dashboard summary stats

Utilities

-
- GET /api/health — Check database connectivity; detailed errors stay in server logs
 - GET /api/seed — Local development only; returns an error in production
 - POST /api/seed — Seed intentionally; production requires SEED_SECRET in the x-seed-secret header

8. SETUP & INSTALLATION GUIDE (Local Development)

Prerequisites (Install These First)

1. Node.js (version 20.9 or higher)

- Download from: <https://nodejs.org/>
- Verify: `node --version`

1. PostgreSQL (version 14 or higher)

- Download from: <https://www.postgresql.org/download/>
- Or use Docker: `docker run -p 5432:5432 -e POSTGRES_PASSWORD=postgres postgres`
- Verify: `psql --version`

1. Git

- Download from: <https://git-scm.com/>
- Verify: `git --version`

Step 1: Clone the Repository

```
git clone
https://github.com/Gauravcodesbyheart/multi-agent-financial-research-research-system.git
cd multi-agent-financial-research-research-system
```

Step 2: Install Dependencies

```
npm ci
```

This installs the exact dependency versions recorded in package-lock.json.

Step 3: Create PostgreSQL Database

```
# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE app_db;
\q
```

Step 4: Configure Environment Variables

Create a .env file in the project root:

```
# Database
DATABASE_URL=postgresql://postgres:yourpassword@localhost:5432/app_db
```

```
# NextAuth
NEXTAUTH_SECRET=your-random-secret-key-at-least-32-chars
NEXTAUTH_URL=http://localhost:3000

# Optional AI configuration (local extraction/retrieval still work without a key)
GROQ_API_KEY=
GROQ_MODEL=openai/gpt-oss-120b
GROQ_MODEL_PRO=openai/gpt-oss-120b
# EMBEDDING_BASE_URL / EMBEDDING_API_KEY / EMBEDDING_MODEL = optional semantic search

# Set when you configure the scheduled recovery worker
CRON_SECRET=

# Optional; set only for deliberate production demo seeding
SEED_SECRET=
```

Optional Groq configuration: If you want model synthesis or embeddings, create a private key at <https://aistudio.google.com/> and set it in .env. Never commit the key. Without it, keyword retrieval and evidence-only local fallbacks remain available.

How to generate NEXTAUTH_SECRET:

```
openssl rand -base64 32
```

Or use: <https://generate-secret.vercel.app/32>

Step 5: Apply the checked-in migrations

For a new, empty database only:

```
npx drizzle-kit migrate
```

If an existing database was initialized with drizzle-kit push, back it up and review/reconcile the pending migrations instead of replaying the initial migration blindly. Do not use automatic schema push in production.

Step 6: Start the Development Server

```
npm run dev
```

Open <http://localhost:3000> in your browser.

Step 7: Optionally Seed Local Demo Data

In a second terminal, use this only for a local demo database:

```
curl http://localhost:3000/api/seed
```

Production GET /api/seed is disabled. For intentional production seeding, POST with SEED_SECRET and the matching x-seed-secret header is required.

Step 8: Login with a Local Demo Account

- `demo@finresearch.ai` or `student@finresearch.ai`
- Local-only password for both: `demo123456`

Change or remove these demo credentials before exposing a public deployment.

9. DEPLOYMENT GUIDE

Option A: Deploy on Vercel + Neon (Recommended)

Provider pricing and limits change. Confirm that your chosen plan supports the Cron frequency and serverless function duration your workload requires.

Step 1: Create accounts

1. GitHub: <https://github.com>
2. Vercel or another Next.js host
3. Neon or another managed PostgreSQL provider

Review each provider's current pricing, region, connection security, Cron, and function-duration limits.

Step 2: Set up a managed PostgreSQL database

1. Go to <https://neon.tech>
2. Click "Sign Up" ! use GitHub
3. Click "New Project"
4. Name it "finresearch-ai"
5. Select region closest to you
6. Click "Create Project"
7. Copy the "Connection String" (starts with postgresql://...)

Step 3: Push your working branch to GitHub

Use the repository's existing Git clone and remote; do not run git init inside it. Push your feature branch with your normal Git workflow, and never commit .env, API keys, database credentials, or private documents.

Step 4: Deploy to Vercel

1. Import the repository and select the intended branch.
2. Configure Preview and Production environment variables separately:
 - DATABASE_URL = private managed PostgreSQL connection string
 - NEXTAUTH_SECRET = strong, unique secret
 - NEXTAUTH_URL = exact public HTTPS origin
 - CRON_SECRET = strong secret for the scheduled job-recovery endpoint
 - GROQ_API_KEY = optional; omit it for evidence-only fallbacks
 - GROQ_MODEL / GROQ_MODEL_PRO = optional model configuration; EMBEDDING_* = optional separate embeddings provider (Groq has no embeddings API)
3. Deploy and verify the health endpoint, sign-in, uploads, research, and reports.

Step 5: Apply database migrations and configure the worker

For a new, empty database, run migrations once from a trusted machine or CI job:

```
DATABASE_URL='postgresql://...managed connection...' npx drizzle-kit migrate
```

If the existing database was initialized using drizzle-kit push, do not replay the initial 0000 migration; back it up and review/reconcile only the missing migrations. GET /api/seed is disabled in production; deliberate seeding requires POST with SEED_SECRET and x-seed-secret. Configure the host's Cron service to call /api/worker/documents with Authorization: Bearer \$CRON_SECRET, then test one queued/recovered job.

Deploy only after you have confirmed the migration history and worker schedule for the intended database.

Option B: Deploy on Railway

Railway may host the app and PostgreSQL; check current pricing, networking, build, and scheduled-task support.

1. Go to <https://railway.app> and create a project from the repository.
2. Configure the environment variables listed above.
3. Confirm the platform's Node.js version, PostgreSQL networking, build command, and start command support the app.

Option C: Deploy on Render (Another Option)

1. Go to <https://render.com> and create a web service from the repository.
2. Create or connect a managed PostgreSQL database.
3. Configure the environment variables listed above and deploy.
4. Confirm that the selected plan supports the required scheduled worker; otherwise configure an external scheduler.

For every host, back up and migrate the intended database using the migration guidance above, then schedule GET /api/worker/documents with Authorization: Bearer \$CRON_SECRET. Use private environment variables, not source files, for credentials. If Groq is enabled, review the provider's data-handling terms before uploading confidential documents.

10. HOW TO USE THE PLATFORM

Step 1: Load Local Demo Data

In local development, sign in and click "Load Demo Data" on the Dashboard (or use the local-only GET /api/seed endpoint). The fixtures include Apple, Microsoft, Tesla, and Amazon sample documents, metrics, and risk flags. They are illustrative; review each source quote and fiscal period before relying on a value. Production GET seeding is disabled.

Step 2: Explore Companies

- Go to "Companies" in the sidebar
- View financial profiles for each pre-loaded company
- Click "Metrics", "Risks", or "Compare" buttons

Step 3: View Financial Metrics

- Go to "Financial Metrics"
- Toggle between Revenue, Margins, Ratios, and Cash Flow charts
- See the detailed comparison table at the bottom

Step 4: Analyze Risks

- Go to "Risk Analysis"
- Filter by severity (Critical, High, Medium, Low)
- Filter by risk type
- Read source quotes and analyst recommendations

Step 5: Run Benchmarking

- Go to "Benchmarking"
- Select 2-8 accessible companies
- Click "Compare Companies"
- View side-by-side charts and comparison table

Step 6: Generate a Report

- Go to "Reports"
- Click "Generate Report"
- Select companies to include
- Click "Generate Report"
- View the AI-compiled analyst-style report

Step 7: Research Agent Chat

- Go to "Research Agent"
- Select a session
- Ask financial questions like:
 - "What is Apple's revenue growth compared to Microsoft?"
 - "What are the biggest risks for Tesla?"
 - "Which company has the best free cash flow generation?"
- Get cited answers with source document references

Step 8: Upload Your Own Documents

- Go to "Documents"
- Click "Upload Document"
- Fill in company name, ticker, document type, fiscal year
- Drag and drop a PDF, DOCX, or TXT file
- Watch its status as durable processing jobs run sequentially; timing depends on file size and configured model calls

11. TROUBLESHOOTING

Groq API key is not configured

- Core local extraction, keyword retrieval, deterministic risk checks, benchmarks, and evidence-only report/chat fallbacks remain available
- Add GROQ_API_KEY from <https://aistudio.google.com/> only if you want model analysis or semantic embeddings
- Keep the key private and restart the app after changing .env

Database connection error

- Make sure PostgreSQL is running
- Check your DATABASE_URL in .env is correct
- Ensure the database exists: CREATE DATABASE app_db;

"Session not found" in Research Agent

- Create or select a research session first
- Go to "Research Sessions" ! "New Session"

Documents stuck in "processing" status

- Processing is sequential and durable; check document status and job errors in the database/logs
- Confirm the /api/worker/documents Cron is scheduled with Authorization: Bearer \$CRON_SECRET
- Extraction and deterministic risk checks have local fallbacks and do not require Groq
- Failed jobs are retried with backoff, then marked failed/partial for manual review

Build errors

Run npm run typecheck to find TypeScript errors

Run npm run lint to find ESLint issues

FINANCIAL TERMS GLOSSARY

Term	Definition
Revenue	Total money earned from selling products/services
Gross Profit	Revenue minus Cost of Goods Sold
Gross Margin	$\text{Gross Profit} / \text{Revenue} \times 100$
EBITDA	Earnings Before Interest, Taxes, Depreciation, Amortization
Operating Income	EBITDA minus D&A
Net Income	Final profit after all expenses and taxes
EPS	Earnings Per Share — net income divided by shares outstanding
Current Ratio	$\text{Current Assets} / \text{Current Liabilities}$ (liquidity measure)
D/E Ratio	$\text{Total Debt} / \text{Shareholders Equity}$ (leverage measure)
ROE	$\text{Net Income} / \text{Shareholders Equity}$ (profitability measure)
ROA	$\text{Net Income} / \text{Total Assets}$
Free Cash Flow	Operating Cash Flow minus Capital Expenditures
10-K	Annual report filed with SEC by public companies
10-Q	Quarterly report filed with SEC

FinResearch AI — Multi-Agent Financial Analysis System

Built with Next.js, Drizzle ORM, PostgreSQL, and Groq AI